

ProjectOne Engineering Handbook

Chapter 2 — Repository Architecture

2.1 Objective

The repository structure must enable fast navigation, low coupling, clear ownership and long-term scalability. Every directory exists for a specific purpose and should never become a generic dumping ground.

2.2 High-Level Structure

```
projectone/  
├── apps/  
├── packages/  
├── infrastructure/  
├── docs/  
├── scripts/  
├── .github/  
└── README.md
```

2.3 apps/

Contains executable applications such as the frontend, backend and future worker services. Applications may depend on shared packages but never on each other directly.

2.4 packages/

Reusable code shared across applications: UI components, types, utilities, SDKs, configuration and common libraries. Keep packages framework-agnostic whenever possible.

2.5 infrastructure/

Deployment configuration, Docker, CI/CD, Terraform (if used), monitoring configuration, secrets templates and operational scripts.

2.6 docs/

Single source of truth for architecture, ADRs, Engineering Handbook, Project Bible and technical specifications. Documentation must evolve together with the code.

2.7 scripts/

Automation scripts for development, migrations, code generation, maintenance and releases. Scripts should be deterministic and idempotent where practical.

2.8 Import Rules

Dependencies always flow inward: applications depend on shared packages; shared packages never depend on applications. Circular dependencies are prohibited.

2.9 Naming Rules

Directories use lowercase with hyphens where needed. Files follow framework conventions. Public modules expose a clear entry point. Avoid abbreviations unless universally understood.

2.10 Ownership

Each major directory has a clearly defined responsibility. If a file does not obviously belong anywhere, create or refine the architecture instead of placing it arbitrarily.

2.11 Anti-Patterns

Do not create:

- miscellaneous folders
- utils containing unrelated logic
- duplicated code across apps
- circular imports
- business logic inside UI components

2.12 Repository Checklist

Before adding a file ask:

- Does this folder own this responsibility?
- Can it be reused?
- Is there a better existing location?
- Does this introduce coupling?
- Will another developer find it intuitively?

Chapter Summary

A predictable repository structure reduces onboarding time, improves AI-generated code quality and prevents architectural drift as ProjectOne grows.